

RovorAI.com

Full Stack Developer Coding Assignment

Position	Full Stack Developer
Expected effort	8 to 12 hours
Tech stack	Next.js + TypeScript, Node.js + TypeScript
Deadline	Thursday evening, 8 October 2026
AI tools	Allowed and encouraged

Overview

This is the coding assignment for the **Full Stack Developer** position at **RovorAI.com**.

The purpose of this assignment is to understand how you approach a real product problem across the frontend, backend, database, external APIs, and application state. We are interested in the quality and structure of your implementation, not just whether the screens work.

You are **allowed and encouraged to use AI tools** such as ChatGPT, Claude, Cursor, GitHub Copilot, or similar tools while completing the assignment. You should understand the solution you submit and be comfortable explaining or modifying your implementation later.

Summary

We want you to build a small project and ticket management application. The home dashboard contains multiple projects. Each project contains tickets and displays a summary of their current status.

Users should be able to create projects, create tickets from the dashboard or from inside a project, open and edit tickets, search and filter tickets, and see changes reflected across the application.

Projects can optionally be connected to a public GitHub repository. For connected projects, the application should retrieve useful repository information through the backend and cache that information.

Required technology

Use **TypeScript** throughout the project. Use **Next.js** for the frontend and **Node.js** for backend functionality. You may implement the backend as a separate Node.js service or use an appropriate server/API architecture within Next.js, provided the frontend and backend responsibilities remain clearly structured.

You may choose the database, libraries, state management approach, styling solution, ORM, caching approach, and other supporting tools.

Requirements

1. Home Dashboard

The home screen should display all projects as large project cards. Each card should provide a useful summary of the project without requiring the user to open it.

Each project card should include:

- Project name and a short description.
- Ticket counts grouped by status, such as Todo, Active or In Progress, and Done.
- A small selection of recent or relevant tickets.
- An option to open the complete project.
- A + action to create a new ticket directly for that project.

The dashboard should also provide an option to create a new project. When projects or tickets are created or changed, the relevant information displayed on the dashboard should reflect the latest saved state.

2. Create Project

Users should be able to create a new project from the dashboard.

A project should contain at least:

- Project name.

- Description.
- Public GitHub repository URL or repository identifier, which may be optional.

Created projects must be persisted in the database and appear in the application after creation.

3. Project Screen

Opening a project should take the user to a dedicated project screen containing the project's information and tickets.

The project screen should allow the user to:

- View the project summary and ticket counts.
- View all tickets belonging to the project.
- Search tickets.
- Filter tickets by status and priority.
- Open an existing ticket.
- Create a new ticket.
- View repository insights when a GitHub repository is configured.

4. Create and Edit Tickets

A ticket should contain at least:

Field	Expected data
Title	Ticket title
Description	Ticket description
Status	Todo, Active or In Progress, Done
Priority	Low, Medium, High
Created At	Creation timestamp
Updated At	Latest update timestamp

Users should be able to create new tickets and edit existing tickets. A ticket may be created from the + action on a dashboard project card or from inside the project itself.

After saving a ticket and navigating back, the project and dashboard should display the updated information without requiring a manual browser refresh.

5. Search and Filters

Users should be able to search tickets and filter them by status and priority. Search should be implemented through the backend rather than relying only on a frontend search over already loaded data.

Search and filters should be able to work together.

6. Backend

Create the backend functionality required to support the application. The exact API design is up to you.

At minimum, the backend should support:

- Retrieving projects.
- Creating projects.
- Retrieving an individual project and its ticket information.
- Retrieving, searching, and filtering tickets.
- Retrieving an individual ticket.
- Creating tickets.
- Updating tickets.

The backend should validate incoming data and return appropriate responses when an operation cannot be completed.

7. Database

Use a persistent database of your choice. At minimum, the data model should represent Projects, Tickets, and the relationship between them.

Seed the application with enough initial data to make it easy to review. Approximately 3 projects and 15 to 20 tickets is sufficient.

8. GitHub Repository Insights

Projects may optionally be connected to a public GitHub repository. For connected projects, display a Repository Insights section containing useful repository information.

This can include information such as:

- Stars.
- Forks.
- Open issues.
- Last updated information.
- Another useful repository metric of your choice.

GitHub information should be retrieved through your backend. The frontend should consume your application's API rather than being built around direct requests to GitHub.

9. Caching

Repository information retrieved from GitHub should be cached for **5 minutes**. Requests made while the cached data is still valid should be able to use the cached result.

You may choose the caching strategy and implementation.

10. Application Experience

The application should provide clear feedback while data is loading, when no relevant data exists, and when an operation cannot be completed.

Visual design is not the primary focus of this assignment. A simple, clean, responsive, and usable interface is sufficient.

11. Scope

You do not need to implement:

- Authentication or user registration.

- Roles and permissions.
- Comments.
- Notifications.
- Real-time collaboration or WebSockets.
- Complex animations.

Submission

Please complete and submit the assignment by **Thursday evening, 8 October 2026**.

Your submission must include:

- **Git repository link** containing the complete source code.
- **README** with complete instructions for running the project locally.
- **Live Vercel deployment link** for the application. If a separate backend hosting service is required, it should be connected and working.
- Any assumptions, known limitations, or incomplete functionality.

README requirements

Your README should briefly explain:

- How to set up and run the frontend, backend, and database.
- The overall architecture of your application.
- How you handle frontend state and application data.
- Your database and data modelling approach.
- Your GitHub integration and caching approach.
- Important technical decisions or trade-offs.
- Which AI tools you used and what you used them for.
- One AI-generated suggestion or implementation that you changed, rejected, or improved, and why.

AI usage is encouraged. The final submission should still be a solution you understand, can explain, and can work with during a follow-up discussion.